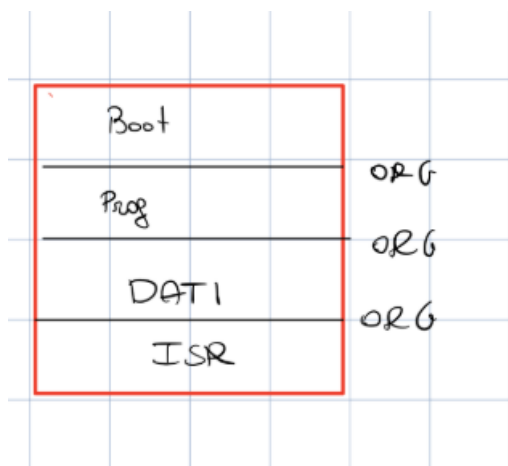


Sommario

1. Gestione delle Interruzioni (Interrupt)	1
3. Il Programmable Interrupt Controller (PIC)	3
Daisy chain	4
4. Modelli di Interruzione: Vettorizzato vs Autovettorizzato	6

1. Gestione delle Interruzioni (Interrupt)

- Esistono architetture di tipo "Memory-mapped" e "I/O mapped".
- Sul BUS viaggiano le linee relative a dati, controllo e stato.
- Esiste un protocollo dedicato del BUS e un protocollo specifico tra la periferica e il mondo esterno.
- Quando si scrive nel registro di controllo vengono utilizzati entrambi i protocolli locali.
- Il protocollo di I/O e il BUS condividono lo stesso tipo di operazione Lettura/Scrittura.

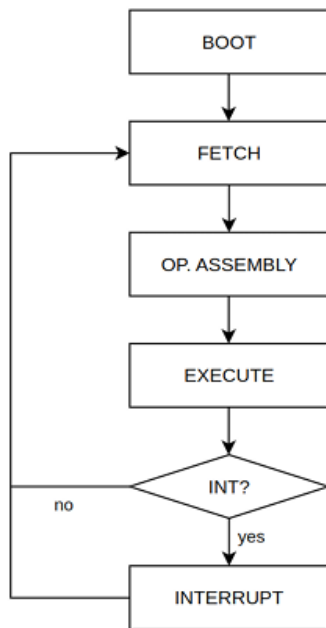


In memoria è possibile condividere diversi elementi allocati tramite direttive ORG, come evidenziato nella tabella.

Le sezioni principali sono: Boot del sistema, Programmi, Dati e le ISR (Interrupt Service Routine): è a tutti gli effetti una programmazione ad eventi.

Se il modello è memory mapped ho anche una parte per i dispositivi di I/O in memoria, mentre se è I/O mapped ho un bus I/O dedicato.

- Quando arriva un'interruzione, il sistema esegue la ISR corrispondente e successivamente effettua un rientro.
- L'indirizzo (ORG) a cui si trova la ISR in memoria può essere fisso o variabile, ma costituisce sempre un riferimento assoluto.
- Il rapporto che lega un evento specifico alla sua ISR è ben definito.



Quando il processore rileva un'interruzione, interrompe la normale esecuzione e salta ad una funzione speciale denominata Interrupt Service Routine ISR.

Tale situazione quindi ferma il sistema per dare priorità alle interrupt. I due registri che richiedono l'obbligo di essere salvati sono *SR (status register)* e *PC (programm counter)* perché potrei, durante la gestione dell'interrupt, modificare qualche registro, e non saprei da che stato ripartire e quale istruzione devo fare.

I registri PC e SR devono essere salvati tramite *un circuito apposito* presente sul processore in ogni caso. La restante parte dello stato di esecuzione viene salvata mediante push sullo stack supervisore perché sono in modalità supervisore da parte della stessa ISR.

Il salvataggio di questi due registri fondamentali **non può avvenire tramite software** perché deve essere un'operazione atomica, non interrompibile da interruzioni di priorità più elevata.

Quando si entra in una ISR, il sistema non sa immediatamente quale periferica abbia generato l'interruzione. Quindi dopo aver salvato i registri, per capire chi ha interrotto, bisogna interrogare il registro di stato di tutte le periferiche.

La prima ha lo stato basso, la seconda basso, la terza alto. Mi ha interrotto la terza.

Il fatto di trovare la prima causa nel registro non esclude che ce ne siano altre pendenti (la prima c'è, la seconda c'è, ecc.). Una volta servita un'interruzione, bisogna controllare le eventuali interruzioni pendenti introducendo il concetto di priorità. Con un solo filo a disposizione, occorre decidere se consentire la ricezione di nuove interruzioni mentre se ne sta già servendo una. Se disabilito le interrupt, significa disabilitare anche le eccezioni interne, ma sono sicuro che nessuno mi interromperà. Se non le disabilito, c'è il rischio che nel mentre ne arrivino altre e devo capire come gestirle.

Avere più fili (es. 2 o 3 per le ISR) migliora la situazione gestendo meglio le priorità, ma aumentarli eccessivamente è controproducente. Questo tipo di approccio va bene principalmente se le periferiche sono poche e hanno tutte lo stesso livello di priorità.

La richiesta di interruzione avviene via Hardware, mentre l'evento e le relative cause sono gestite tramite Software.

Per evitare accavallamenti, le periferiche possono essere collegate tra loro con un meccanismo a catena: in modo che la periferica *iesima* può interrompere se e solo se quelle precedenti non hanno interrotto.

Il processo di gestione di un'interruzione generale è il seguente:

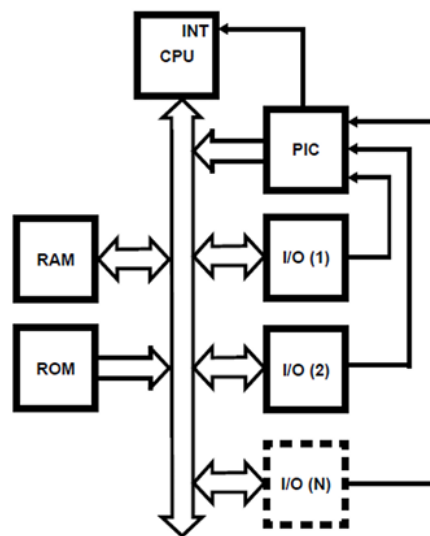
- Esecuzione normale;
- Servizio dell'interruzione:

- Salvataggio contesto Hardware;
- Identificazione dispositivo richiedente
- Salto all'entry point della ISR;
- Salvataggio del contesto Software;
- Servizio dell'interruzione;
- Ripristino contesto software;
- Ripristino contesto Hardware;
- Ripresa esecuzione normale.

Occorre definire delle priorità nel caso di interrupt molteplici concomitanti, occorre gestire il caso degli interrupt innestati, e infine occorre gestire la presenza di più periferiche che condividono la stessa linea fisica di interruzione.

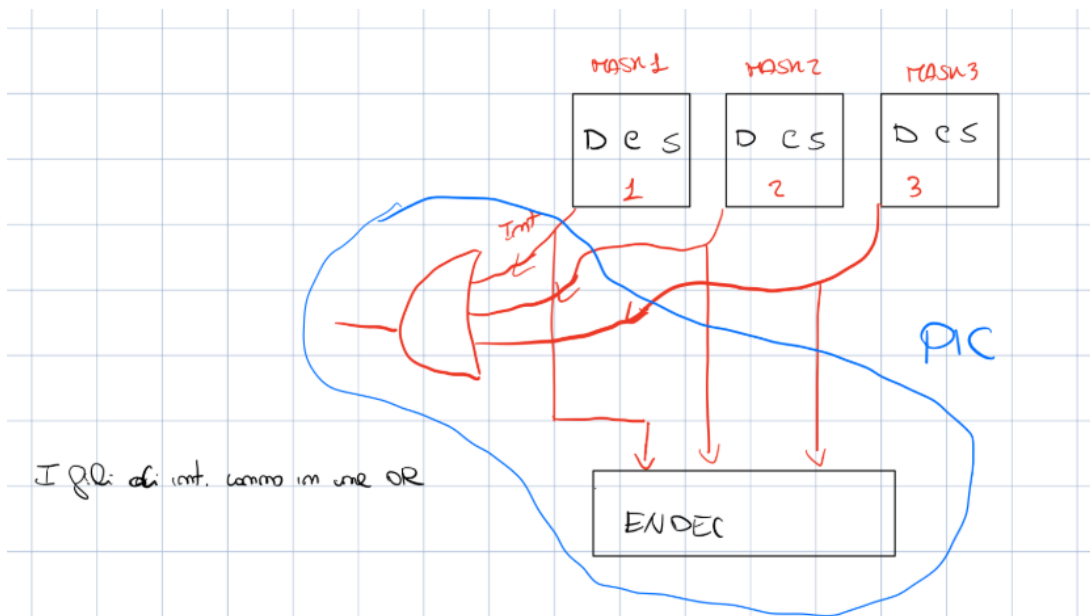
3. Il Programmable Interrupt Controller (PIC)

Microcomputer system design requires that I/O devices such as keyboards, displays, sensors and other components receive servicing in an efficient manner so that large amounts of the total system tasks can be assumed by the microcomputer with little or no effect on throughput. *The most common method* of servicing such devices is the Polled approach. This is where the processor must test each device in sequence and in effect "ask" each one if it needs servicing.



I sistemi moderni, gestendo tantissime periferiche in cui il processore aspetta direttamente gli eventi, non utilizzano l'approccio di polling software precedentemente descritto.

Ovviamente quando compro una periferica, questa non sa di essere stata comprata, ne tantomeno sa che verrà montata insieme ad altre periferiche.



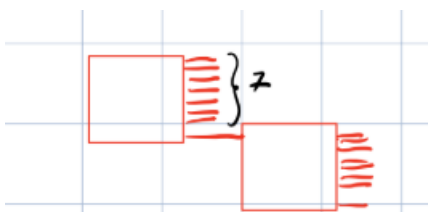
Ho diverse periferiche, e come detto fino ad ora hanno dei fili sull'interfaccia lato processore. Tra questi c'è anche il filo di interrupt. *Questi fili confluiscono in una OR* in modo da sapere se c'è stata un'interruzione. Inoltre, voglio sapere chi mi ha interrotto. E quindi c'è la presenza di un *encoder di priorità*, che darà in uscita 3 fili ovvero i bit che identificano chi mi ha interrotto. Il Pic ha al più 8 periferiche in ingresso, per questo in uscita 3 fili.

La periferica 2 ha alzato il bit interrupt, il risultato della OR è 1. Il processore sa che si è verificata un'interruzione.

Il PIC permette anche di settare delle maschere per ogni periferica, tramite l'interupt mask register. Questo permette di escludere e disabilitare in modo selettivo o totale le interruzioni da periferiche che possono dare fastidio. Lo scopo del PIC è quello di comunicare tramite la or se c'è stata un'interruzione e fornire l'identificativo di chi mi ha interrotto così da andare nella ISR corrispondente.

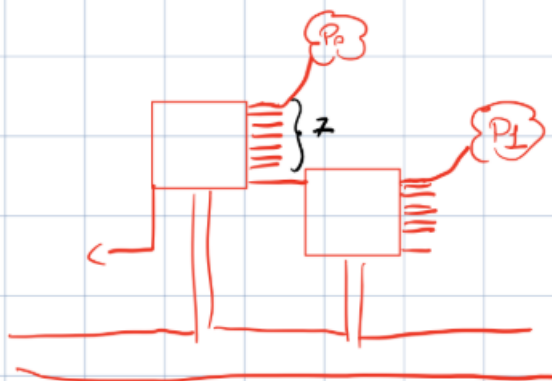
Daisy chain

Un PIC può avere un numero definito di ingressi (es. 8), ma se vogliamo più periferiche, possiamo montare in cascata più PIC collegando l'ultimo filo del PIC più alto all'uscita del pic più basso.



I PIC in cascata devono comunicare senza generare conflitti, scrivendo sul BUS interno l'identificativo e il segnale di interruzione.

Come fanno i PIC e interagire con il processore?



Abbiamo questi due PIC collegati in cascata. Su quello più alto abbiamo 7 periferiche P0. L'ultimo filo è collegato al secondo PIC e abbiamo altre 8 periferiche di tipo P1. Se la or del PIC più alto da in uscita 1 significa che c'è stata un'interruzione o dalle sue 7 periferiche, oppure perché l'ottavo filo si è alzato e cioè la or del secondo PIC era alto. Ora però se i due pic scrivessero insieme sul BUS, sarebbe un casino. Quindi se il pic più in alto si accorge che ha segnalato l'interruzione a causa del ottavo filo, capisce che non deriva dalle sue periferiche e quindi non scrive niente sul BUS, passando la palla agli altri PIC. Il PIC che capisce che l'interruzione è dovuta ai suoi 7 ingressi, scriverà l'identificativo sul BUS.

Questo identificativo, che fuoriesce dal priority encoder, dove va? Va sul BUS DATI ed è a 8 bit perché corrisponde all'indirizzo del vettore. $2^8=256$ da 0 a 255 ovvero la dimensione della tabella dei vettori.

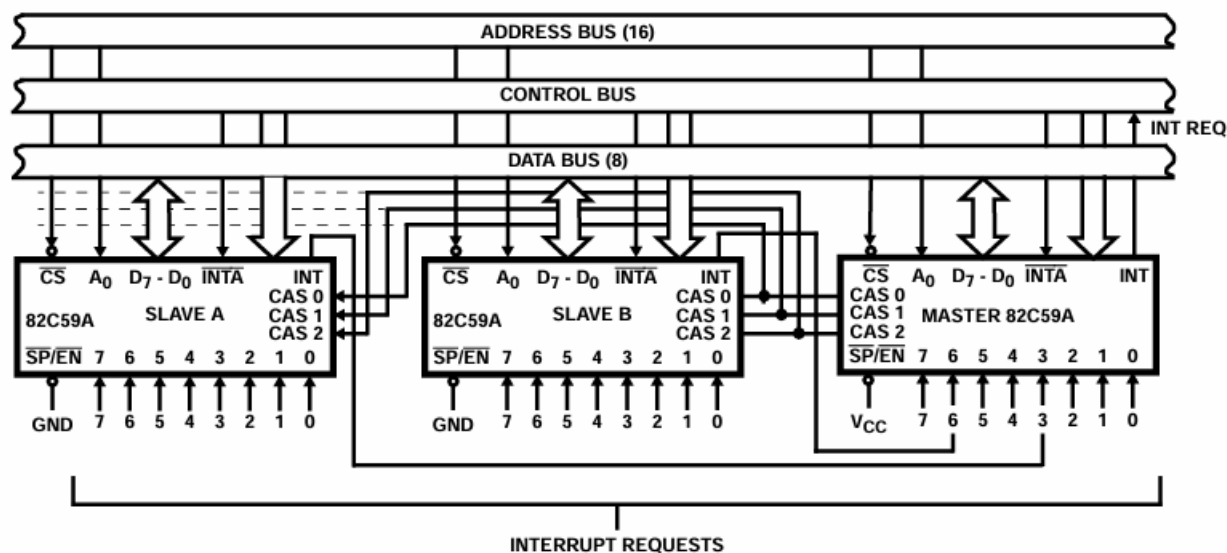


FIGURE 11 CASCADING THE 82C59A

Nel caso di PIC in cascata vediamo che c'è il PIC master che è quello di livello più alto, esso è collegato a VCC e la sua linea di interruzione confluisce direttamente sul bus di controllo del processore nella INT REQ. Le linee di interruzione degli slave (che sono collegati a massa così capiscono di essere slave), confluiscono nelle 8 linee di interruzione IRQ0-IRQ7 del master. Nell'esempio un PIC è collegato alla linea 3 e un PIC alla linea 6.

Inoltre ci sono 3 segnali CAS0-1-2 (CAS sta per cascade) che vengono utilizzati per la comunicazione con gli slave. Il segnale di ritorno della CPU INTA* confluisce sia nel PIC master che nei PIC slave. Queste linee servono per dare il chip select agli slave in modo da coordinarsi per scrivere sul bus dati l'identificativo della linea interrompente.

Quando la ISR termina invia due segnali di fine interruzione (EOI): uno per il master e uno per lo slave.

Un dispositivo attaccato allo Slave numero 2 genera un'interruzione. Lo Slave 2 alza la mano col Master; il Master alza la mano con la CPU. Quando la CPU risponde dando il via libera (con il segnale INTA), si crea una situazione di stallo: come fa lo Slave numero 2 a sapere che la CPU ha dato l'ok proprio a lui e che deve mettere il suo Vettore sul bus dati, visto che questo segnale arriva a tutti? Gli Slave sono "ciechi", non sanno cosa fanno gli altri Slave. Se tutti provassero a scrivere sul bus dati al primo INTA, andrebbe tutto in cortocircuito.

Con 3 fili (CAS0, CAS1, CAS2), il Master può generare in binario i numeri da 0 a 7. La CPU invia due volte il segnale INTA:

- Il primo INTA: La CPU invia il primo segnale di Acknowledge. Questo arriva solo al Master. Il Master sa che la richiesta originale proveniva dal suo pin numero 2 (dove c'è collegato lo Slave). Quindi, il Master prende i tre fili CAS e vi applica le tensioni per formare il numero 2 in binario: CAS2=0, CAS1=1, CAS0=0. Queste tre linee elettriche arrivano a tutti gli Slave contemporaneamente. Ogni Slave legge il codice binario e lo confronta con il proprio nome. Lo Slave 1 legge 010 e dice: "Non sono io", e rimane in silenzio. Lo Slave 3 legge 010 e dice: "Non sono io", e rimane in silenzio. Lo Slave 2 legge 010 e dice: "Sono io! È il mio turno!".
- Il secondo INTA: La CPU lancia un secondo impulso di Acknowledge. A questo punto, solo lo Slave 2 (che è stato "svegliato" e autorizzato dalle linee CAS) apre il suo buffer e spara il suo Vettore a 8 bit sul bus dati della scheda madre.

4. Modelli di Interruzione: Vettorizzato vs Autovettorizzato

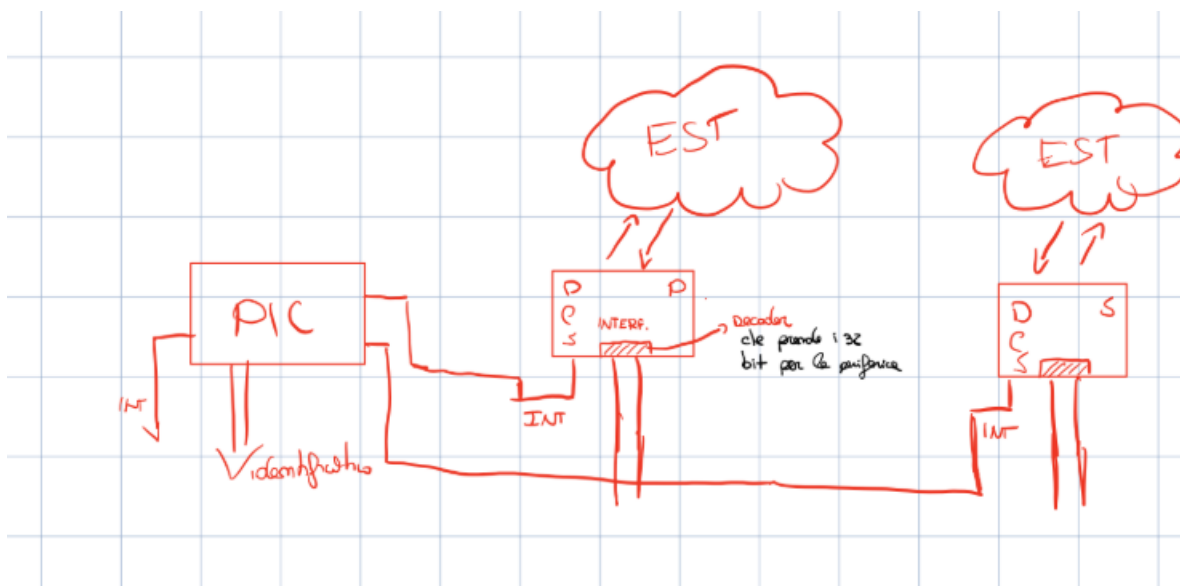
Il 68 k ha due soluzioni che può adottare:

- Sistema vettorizzato

-Sistema autovettorizzato

Modello Autovettorizzato: L'identificativo viene fornito direttamente tramite fili specifici dedicati, che sono 3 per gestire soltanto 8 periferiche. Grazie ai 3 pin, collegati direttamente all'encoder, sappiamo chi degli 8 mi ha interrotto. In questo caso gli 8 livelli corrispondono alle priorità.

Modello Vettorizzato: posso avere molti più oggetti. L'identificativo codificato dal PIC (su 8 bit) viene letto dal processore sui 32 bit delle linee dati del BUS. Se l'encoder fornisce 8 bit, posso avere 64 periferiche, quindi un sistema formato da tanti pic in cascata.



Mondo esterno è una periferica, può essere una stampante, un sensore ecc. si interfaccia con la CPU tramite l'interfaccia come abbiamo sempre detto. Queste interfacce hanno i registri DATO STATO CONTROLLO e possono essere parallele o seriali. Inoltre le interfacce hanno un decoder (Adapter) che è quello che porta con se la chiave di 32 bit per capire se un segnale è diretto a quella periferica. Con 30 bit ho l'indirizzo dove si trova la periferica, con i 2 bit meno significativi capisco se è registro dato, stato, controllo.

Il processore una volta che riceve l'identificativo per sapere quale ISR selezionare fa il seguente calcolo: $\text{BASE_ISR} + \text{Identificativo} * 4$

La base è fissa, cioè è l'inizio della tabella delle ISR, l'identificativo è quello scritto sul BUS. Moltiplico per 4 poiché ogni vettore è di 32 bit, quindi se ho 0 come identificativo rimango a Base, se ho 1 devo fare base + 4, cioè la long successiva e così via.

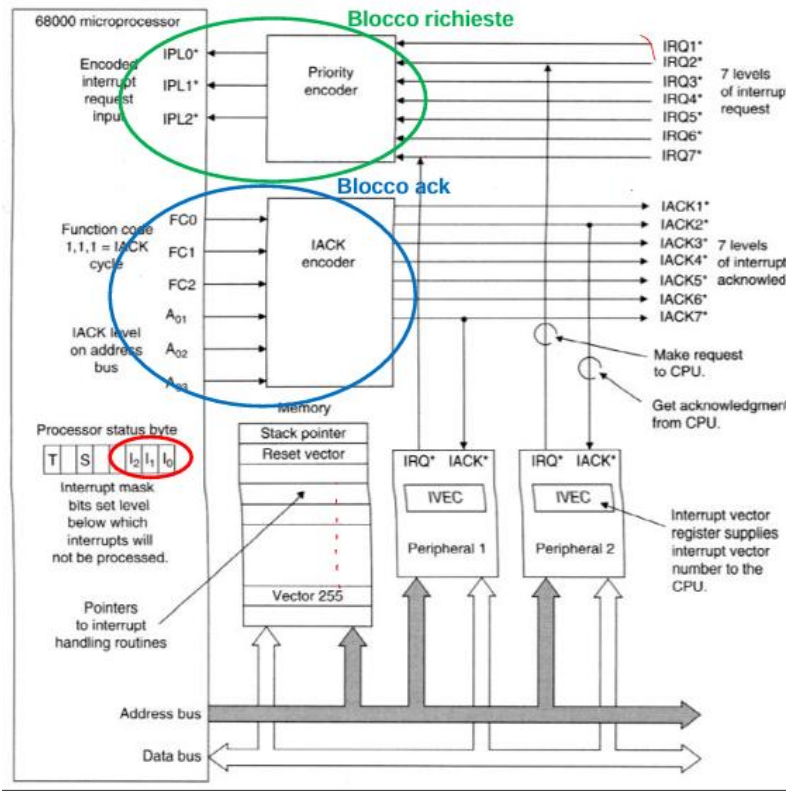
L'identificativo che il processore aspetta viene chiamato vettore che quindi viene usato per identificare la periferica. Identificare la periferica significa individuare la corretta ISR che può gestire l'interruzione. La tabella delle ISR parte dall'indirizzo 0 e contiene 256 locazioni di 4 byte.

Vector Number(s)	Range	Description
0	0	Reset (SSP)
1	1	Reset (PC)
16-23	16-23	Unassigned, Reserved
25	25	Level 1 Autovector
31	31	Level 7 Autovector
32-47	32-47	Trap #0-15 Instructions
64-255	64-255	User Device Interrupts

Qui notiamo che da 25 a 31 ci sono 7 livelli autovector. Questo perché se il sistema è autovettorizzato, la base non è 0, ma si parte da un'altra parte di memoria che coincide con 25 e si sfruttano direttamente i 3 pin per capire l'offset. Per sapere a che indirizzo si trova la base facciamo $25 * 4 = 100$ e poi convertiamo in decimale ovvero \$64

5. Gestione delle interruzioni e passi sul M68k

In questo disegno ci sono due periferiche collegate a due linee differenti. Una collegata a IRQ2 e una collegata a IRQ7.



- 1) Una o più periferiche asseriscono il segnale di interruzione IRQ
- 2) Il più alto livello di richiesta viene codificato dal priority encoder che genera 3 bit passati al processore sui pin PL2-0
- 3) Il processore confronta la priorità della richiesta con il valore contenuto negli interrupt mask flag. Se l'interruzione può essere servita allora invia un ACK sulla linea corrispondente. L'ack serve per dire alla periferica ho accettato/non ho accettato. Se ho una più prioritaria non posso accettare.
- 4) Il processore identifica il dispositivo da servire e invoca il rispettivo handler. questa fase cambia in base al sistema se è vettorizzato oppure autovettorizzato

Sistema autovettorizzato

- 1) Dopo aver ricevuto l'ACK, il dispositivo asserisce un segnale (VPA) per segnalare che usa gli autovettori
- 2) Il processore carica direttamente il vettore corrispondente alla linea interrompente.

Sistema vettorizzato:

- 1) Dopo aver ricevuto l'ACK il dispositivo invia il proprio vector number sul bus dati ed asserisce un segnale DTACK
- 2) Il processore carica il vettore richiesto.

Il priority encoder non è un PIC, una cosa più semplice, però fa una cosa prioritaria. Se ho due richieste fa andare avanti quella prioritaria.

Questo schema si modifica se viene utilizzato il PIC per poter inserire più dispositivi.

Una cosa importante è che potrei avere più oggetti attaccati alla stessa linea. A questo punto se mi arriva un'interruzione da una linea, devo andare in polling sul registro di stato a vedere chi di quelli attaccati alla stessa linea mi ha interrotto.

Lezione 20 Marzo

Come funziona il sistema delle interruzioni vettorizzate. Ci sono tre aspetti diversi: 1 ricevere l'interruzione, un altro è gestire le priorità, un altro è servire l'interruzione. La gestione delle

interruzioni con il PIC garantisce 3 cose: che l'interruzione arrivi, che possa gestire la priorità, che possa identificare l'interruzione, ma non che possa servirla, perché serve un ISR cioè un programma, una routine.

Da un lato abbiamo le interruzioni esterne e dall'altro lato abbiamo le eccezioni che possono essere sollevate da due cause diverse: una legata alla possibilità di avere degli errori come ad esempio uso di indirizzi illegali, oppure istruzioni illegali etc. Il sistema non può andare avanti e deve essere gestito.

Poi ci possono ulteriori cause che richiedono di passare da stato utente a stato supervisore, come divisione per zero, uso dell'istruzione TRAP.

Il processore Motorola 68000 utilizza una vector table, che occupa 256 locazioni di 4 byte ciascuna nella porzione di memoria riservata allo stato supervisore, a partire dal byte di indirizzo \$0000 fino al byte di indirizzo \$03FF

- Ogni locazione della vector table, detta exception vector, contiene l'indirizzo di un exception handler

- Ogni locazione è identificata da un indice di 8 bit che rappresenta univocamente il tipo di eccezione corrispondente ed è detto vector number

Ci sono degli entry point diversi. Ad esempio troviamo i 7 livelli dati per l'autovettore, cioè quando il sistema è autovettorizzato.

Da 64 a 255 è riservato di istruzioni vettorizzate. Ci sono anche una serie di operazioni. La linea 5 che si trova all'indirizzo 14 c'è la divisione per zero. Quindi quando c'è un errore, produce un eccezione. In hardware va nella linea 5 ovvero all'indirizzo 14 esadecimale e preleva la ISR che serve a gestire la divisione per zero. Quindi nel vettore dobbiamo mettere l'indirizzo della ISR in modo che quando va a prendere l'indirizzo il sistema converge. Quando siamo nella divisione per zero, da qualche parte ho fatto una ORG ad esempio 7000 ci sarà il programmino per gestire la divisione per 0 e quindi nel vettore scrivere 7000. Devo andare a scrivere nel vettore delle interruzioni l'indirizzo della ISR che gestisce la divisione per zero. Quando il sistema parte, la prima cosa da fare, prima di abilitare le interruzioni, è scrivere tutti gli indirizzi delle ISR.

vector number (Decimal)	address (Hex)	assignment
0	0000	RESET: initial supervisor stack pointer (SSP)
1	0004	RESET: initial program counter (PC)
2	0008	bus error
3	000C	address error
4	0010	illegal instruction
5	0014	zero divide
6	0018	CHK instruction
7	001C	TRAPV instruction
8	0020	privilege violation
9	0024	trace
10	0028	1010 instruction trap
11	002C	1111 instruction trap
12*	0030	not assigned, reserved by Motorola
13*	0034	not assigned, reserved by Motorola
14*	0038	not assigned, reserved by Motorola
15	003C	uninitialized interrupt vector
16-23*	0040-005F	not assigned, reserved by Motorola
24	0060	spurious interrupt
25	0064	Level 1 interrupt autovector
26	0068	Level 2 interrupt autovector
27	006C	Level 3 interrupt autovector
28	0070	Level 4 interrupt autovector
29	0074	Level 5 interrupt autovector
30	0078	Level 6 interrupt autovector
31	007C	Level 7 interrupt autovector
32-47	0080-00BF	TRAP instruction vectors**
48-63	00C0-00FF	not assigned, reserved by Motorola
64-255	0100-03FF	user interrupt vectors

Nota: I vettori 0 e 1 sono associati al reset e contengono lo SSP e il PC da caricare dopo il reset

Nota: Alcuni vettori fanno riferimento agli interrupt (cioè le interruzioni generate da dispositivi esterni)

Se prendiamo il livello 5 per le interruzioni auto vettorizzate, significa ad esempio scrivere nella riga ORG 8700 che è dove si trova la ISR per gestire la periferica identificata dal numero 5.

Ogni volta che metto una periferica ho una doppia promessa: una che metto la org che dice dove è la ISR, e un'altra che nel vettore delle interruzioni metto quell'indirizzo, altrimenti non combacia niente.

È un meccanismo hardware. Non possiamo scegliere dove mettere la tabella. Il fatto di avere il vettore delle interruzioni consente di allocare le ISR dove vuoi tu e quindi fare un indirizzamento indiretto, con il vincolo che il vettore sta lì e non lo muovi, ma gli indirizzi delle ISR sono allocabili dove vuoi.

Se vediamo indirizzo 0 e indirizzo 4 quelli sono la fase di bootstrap, quelli che servono per partire con il sistema. A tutti gli effetti c'è tutto il funzionamento di una macchina.

Quando dividi per 0, il processore non ha il PIC ovviamente. Allora va in memoria alla locazione 0014 e lì c'è scritto l'indirizzo della ISR e lo carica nel program counter. Il pic non serve perché la causa d'interruzione è interna, non esterna che non conosce. Quella interna la identifica.

Quando dividiamo per zero, cosa succede nel programma c? il programma c viene tradotto in assembly, eseguite una alla volta dall'assembler. Genera un' interruzione, va nella vector table, carica 7000 nel program countere fa girare una routine.

Stato utente e stato supervisore

- **Il Mondo Utente (S=0):** Usa il suo blocco per gli appunti (lo stack **USP o A7**) e non può toccare l'hardware.

- **Il Mondo Supervisore (S=1):** Usa un blocco per gli appunti completamente diverso e nascosto all'utente (lo stack **SSP o A7 primo**) e può fare tutto.

Supponiamo di essere in stato utente e di voler fare questa istruzione per accendere il Bit S (il bit 13) dello Status Register:

`MOVE.W #$2700, SR`

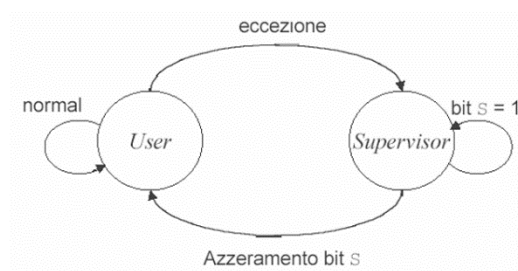
Prima di eseguire qualsiasi istruzione che tocca lo Status Register, il processore controlla in che stato ti trovi. Vede che il tuo Bit S attuale è 0 (sei un utente normale) e blocca tutto.

L'istruzione non viene eseguita. Al contrario, la CPU solleva immediatamente una delle eccezioni interne più gravi: **la Privilege Violation** (Violazione di Privilegio).

L'hardware ti strappa letteralmente il controllo dalle mani, passa in stato Supervisore da solo e chiama il Sistema Operativo dicendogli: "Attenzione, questo programma ha cercato di ottenere permessi che non ha". Il Sistema Operativo, come risposta, chiude forzatamente il tuo programma (è l'equivalente del famoso "Segmentation Fault" o "Access Violation").

In sintesi, la gestione delle eccezioni comporta i seguenti passi:

- Il processore esegue una copia temporanea dello SR e poi setta il bit S per **passare nello stato supervisore**
- Il processore determina il vector number dell'eccezione (con meccanismi interni oppure mediante una comunicazione con dispositivi esterni nel caso delle interruzioni)
- Il processore (tranne che per l'eccezione provocata dal reset) salva il PC e la copia che aveva fatto dello SR sul SSP e inserisce nel PC l'indirizzo della ISR
- Il processore effettua la fetch dell'istruzione identificata dal vector number e passa ad eseguire l'exception handler.
- Dopo aver servito l'eccezione, il processore resetta il bit S e ripristina i registri salvati, per poi ritornare al programma interrotto



le istruzioni per il cambiamento di stato sono istruzioni privilegiate. Il passaggio da livello utente a quello supervisore può avvenire nelle seguenti modalità:

- 1) Interrupt da una periferica esterna
- 2) Evento di errore nell'esecuzione di un'istruzione (eccezione)
- 3) Istruzione TRAP: richiesta di interrupt software

Viceversa la transizione da stato supervisore a stato utente può avvenire:

- 1) Tramite RTE: l'eccezione termina e il processore torna allo stato utente
- 2) Azzerando il bit S attraverso un'operazione esplicita di MOVE, oppure di ANDI con lo SR

Le istruzioni privilegiate nel MC68000 sono:

- STOP: non può essere concessa ad un utente perché può bloccare i processi di altri utenti
- RESET: invia un segnale di reset ai dispositivi esterni. Potrebbe resettare i dispositivi usati da altri utenti;
- RTE: ritorno da un eccezione. L'eccezione ha il privilegio di supervisor
- MOVE to SR/ANDI to SR/ORI to SR: un utente non può modificare lo SR

Interrupt software o eccezioni software (TRAP): invece di fare un codice operativo che non possiamo fare, chiediamo a qualcuno di poterlo fare. Supponiamo che voglio leggere una locazione di memoria che corrisponde all'accesso al bancomat e non lo posso fare. Se faccio l'operazione di lettura vengo portato fuori dal sistema, cioè parte una ISR che dice non puoi fare questa cosa.

Allora devo chiedere al sistema operativo, lo fai tu per me? È meglio specificare che questo qualcuno non è detto che sia il sistema operativo, è un sistema che ha più potere di noi. Molte volte coincidono.

Tramite il codice operativo TRAP il sistema passa da stato utente a supervisore e può fare tutto. Il codice che volevo fare non è più illegale.

A meno che anche la ISR che gestisce la TRAP non fa un codice corrotto. A quel punto bisogna gestire un codice corrotto fatto dallo stato supervisore e sono guai, non è facile. Sul Motorola 68000, se si verifica un'eccezione grave (come un errore di indirizzamento o di bus) mentre il processore è già all'interno di una routine di gestione delle eccezioni, si verifica quello che in architettura viene chiamato **Double Fault** (Doppio Guasto). Dato che il sistema non può fidarsi nemmeno del codice di recupero, il processore si arrende, smette di processare istruzioni e passa in uno stato di **HALT** hardware. L'unico modo per uscirne è un reset fisico della macchina (è l'equivalente a basso livello del *Kernel Panic* in Linux o della *Blue Screen of Death* in Windows).

L'indirizzo della ISR dell'istruzione TRAP si trova al vettore numero 7. Cioè all'indirizzo 28 decimale ovvero \$1C

La ISR dovrebbe capire cosa voglio fare, cioè perché l'ho chiamata. La TRAP è un solo codice operativo, ma come fa a sapere io cosa voglio fare? Potrei voler leggere un carattere, scrivere, stampare, aprire un file e tante altre cose.

Normalmente in questo caso ci vuole una convenzione. Nel registro D0 ti metto un numero, che serve a identificare perché ti ho chiamato, cioè è il motivo per il quale ti ho chiamato. Quando arriva l'interruzione voluta da me con TRAP, passo da stato utente a supervisore e il programma inizia. Lui deve capire prima di tutto perché lo abbiamo chiamato. Ora succede che magari io volevo far stampare un vettore e quindi gli devo dare anche l'indirizzo e la dimensione cioè dei parametri. Li posso mettere in A0 e D1. Quindi in generale in base al codice che gli passo tramite D0, sa benissimo che si aspetta certi parametri e dove io li ho messi.

Quando facciamo cin e cout in c++, non è mai vero che siamo noi a stampare un carattere. Ma si lancia un'eccezione e si passa da stato utente a stato supervisore. In qualche modo il traduttore gli ha messo l'identificato tipo in D0 dove c'è la convenzione e stampa.

Questo è un modo diverso dagli altri due (eccezione e interruzione), perché il primo è un evento di errore interno (accesso a indirizzo illegale, istruzione non consentita, divisione per zero etc), quello esterno è dato da una periferica esterna. Questo è l'unico modo per passare da stato utente a stato supervisore.

Nella vector table ci entro per l'interruzione che ho avuto. Dentro alla ISR poi mi serve D0 per dire tu volevi passare in stato supervisore, ma per quale motivo? È come se nella ISR ci fosse un case D0.

Supponiamo di fare un operazione printf, devo attivare una periferica, e non la posso attivare, perché la voglio far attivare dal sistema operativo.

- 1) La prima operazione è quella che mi consente di fare una eccezione/interrupt software
- 2) Questo fa attivare un programma che sta a 10000 ad esempio (ISR)
- 3) La ISR vede il valore che sta in D0 e capisce che voglio attivare la periferica grazie al numero scritto in D0

- 4) A questo punto attiva la periferica
- 5) La periferica prima o poi finisce. Quando finisce genera un'interrupt esterna che può essere vettorizzata o autovettorizzata.

In Asim questa cosa non la vediamo perché siamo sempre in stato supervisore.

Supponiamo che debba scrivere un driver per un'interruzione che riceve 100 caratteri. Cosa fa il main e la ISR se voglio leggere 100 caratteri.

il main è il programma di stato supervisore attivato dal codice che mi fa passare nello stato. Devo attivare una periferica.

Il vettore che vogliamo stampare si chiama vett. lo mette nel registro di dato della periferica. La periferica non è partita ancora. Deve mettere qualcosa nel registro di controllo. Ha messo il dato, il controllo per lui ha finito.

Il main deve anche dire da qualche parte voglio scrivere 100 caratteri. Quando arriva la ISR, la periferica che ha interrotto ha finito di stampare un carattere. Nella ISR dobbiamo andare a trasmettere un altro carattere.

ISR

$N=n-1$

Se n diverso da zero

$Vet = Vet + 1$ lo metto in Dato

In un sistema semplice che non gestisce descrittori ecc non facciamo un ciclo for. Non vediamo il registro di stato perché se ci ha interrotto e quella è linea sicuramente ci ha interrotto lei. Nel main prima del controllo conviene pulire lo stato per evitare che rimanga qualcosa vecchio.

Questo va bene perché sto facendo un sistema embedded, se dovessi fare un sistema operativo dovrei mettere **dei descrittori** per dire chi ha aperto la risorsa, che cosa sta facendo ecc. Ci sono mille programmi aperti. Se arriva un'interruzione dalla stampante, di chi è quel documento? Di Word o del Browser? Per questo servono i Descrittori (strutture dati in memoria che tracciano chi ha richiesto quale risorsa, a che punto è, e se ha i permessi per farlo). Esistono descrittori di processo (PCB) che includono il PID, lo stato del processo e altro e i descrittori delle periferiche (Device Control Block)

La ISR la metto in un indirizzo che ho deciso io, e quell'indirizzo lo metto nel vector table ad esempio al livello 1 autovettorizzato se è autovettorizzato. Quindi fisicamente collego la periferica a IRQ1. Se volessi cambiare, dovrei fare un movimento hardware, cioè collegarlo su un altro filo.

Questo sistema è un po' strano. Ogni volta che metto il dato pulisco lo stato e attivo il controllo.

Potrebbe essere fatto in modo automatico. Ogni volta che scrivo il dato, in modo automatico il sistema resetta lo stato e attivo il controllo. Se la periferica è più intelligente lo fa lei.

Una periferica intelligente inoltre deve dare una modalità di modo e dire se lavora in polling o con le interrupt.

Se la periferica è parallela o seriale, la ISR non cambia. È un problema della periferica non mio. Se è parallela esce in parallelo, se è in serie uscirà seriale, nel senso che poi ci pensa la periferica a inoltrarlo, io mi occupo solo di mettere nel registro il dato che voglio mandare o di leggerlo, sono dalla parte del processore e voglio leggere byte sempre.

Modalità TRACE

Modalità Trace (Bit T): Impostando il bit T nello Status Register, la CPU genera automaticamente un'eccezione interna dopo ogni singola istruzione. Questo forza il passaggio in Supervisore permettendo a un software Debugger di mostrare i registri passo dopo passo (esecuzione step-by-step).